

Wentworth Institute of Technology

COMP3450 – Parallel and Distributed Programming

Spring2026

From this lab, I present two graphs, and some pictures of the Mandelbrot set, one with black and white color, one with 16-bit color striations, and one with the Bernstein color pallet. The images viewed are of 24000x16000. The data collected is from images that are 60060 x 40040, which is the largest size I could get to run, and it runs up against the 64kx64k limit on the number of pixels in a jpeg image. The cpp code is not fast enough to render the images, so I did not create one that large. I imagine it could also be parallelized but I decided that was out of scope.

The two graphs below show the schedule differences when ranks increase and when thread count increases. The first graph shows rank increase. There was some bug I could not find that caused the 3 rank to fail multiple times, so the data there is poor. As we would expect, the data does decrease as rank increases. If there is further optimization to do when just increasing rank, as the parallelization is naïve, using modular arithmetic to load balance by row. We do not see the typical decrease and then increase because the problems are so large, and at this scale, more ranks would help. There was an overflow error that I also did not solve that prevented 10, 11, and 12.

The second graph shows the increasing thread count on 2 ranks and on 8 ranks. On 2 ranks, the trial with only 10 threads timed out, but when we look at the comparison over 20 threads, we can see a significant difference. The 8 ranks almost half the time in comparison to the 2 ranks at 20 threads. As the thread count increases, the gains in efficiency become smaller. With 2 ranks and 80 threads per rank for a total of 160 threads, we approach 8 ranks at 40 threads per rank for a total of 320 threads near the 20 second performance time. Given enough threads per rank, the efficient work done by each rank is greater. With less overall compute, the computer does a similar job. This is because the OMP call is further optimized by dynamic scheduling with small chunks. This dynamic scheduling accounts for a higher performance due to the nature of the problem, where some portions of the Mandelbrot set are empty while others are incredibly full. Threads also boast a faster networking speed so the higher thread count reduces the overall communication costs.





